

Cos'è il PageRank?

- **Definizione:** PageRank è un algoritmo sviluppato da Larry Page e Sergey Brin nel 1998, alla base del motore di ricerca Google. Assegna a ogni pagina web un punteggio di importanza basato sulla struttura dei link.
 - **Modellazione del Web:** Rappresentazione come grafo diretto $G = (V, E)$, con V insieme dei nodi/pagine web ed E insieme dei link tra le pagine.
 - **Principio Cardine:** Una pagina è tanto più importante quanti più link in ingresso riceve e quanto più essi partono da pagine importanti.
-

Cos'è il PageRank?

LA FORMULA DEL PAGERANK

$$PR(i) = \frac{1-d}{N} + d \cdot \sum_{j \in In(i)} \frac{PR(j)}{L(j)}$$

PR(i) — Punteggio del nodo

Il valore di PageRank del nodo i che si vuole calcolare. Probabilità che il surfer arrivi qui.

(1-d)/N — Teleportation

Probabilità di "teletrasporto" casuale su qualsiasi nodo. Garantisce convergenza globale.

d — Damping Factor

Fissato a 0.85. Probabilità che il surfer segua un link. Previene spider traps e sink.

$\sum_{j \in In(i)}$ — Sommatoria

Somma su tutti i nodi j che puntano al nodo i. In(i) = insieme degli in-link.

PR(j) / L(j) — Contributo ponderato del nodo sorgente

PR(j) = PageRank del nodo sorgente j che linka verso i. L(j) = out-degree di j (numero di link uscenti). Ogni link "vale" PR(j)/L(j); il PR si distribuisce equamente tra i destinatari.

d = 0.85 → **85%**

1-d = 0.15 → **15%**

Dati impiegati

Scala del Grafo

4.847.571

NODI
(Pagine Web)

68.993.773

ARCHI
(Link)

~14.23

ARCHI / NODO
(Fattore di densità)

⚡ Perché questa scala?

Con grafi ridotti, la comunicazione di rete domina l'elaborazione, vanificando inoltre l'effetto di eventuale asincronia.

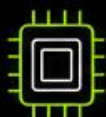
Dataset principale: **soc-LiveJournal1** caricato a runtime

· 4.8M nodi · 69M archi

Ambiente di **esecuzione**



CARATTERISTICHE DEL CLUSTER



NODI COMPUTAZIONALI (x4)

- 2 processori Xeon (3.40 GHz)
- 8 core fisici per processore
- 16 core fisici totali per nodo
- RAM 64 GB
- Disco 1 TB
- Hyper-Threading su alcuni nodi



NETWORK

- InfiniBand - 56 Gb/s



TOPOLOGIA

- Likely Fat-Tree



SISTEMA OPERATIVO

- Linux

Dalla Baseline al Parallelo

Analisi della versione originale e criticità iniziali.

Il Punto di Partenza

Versione Sequenziale

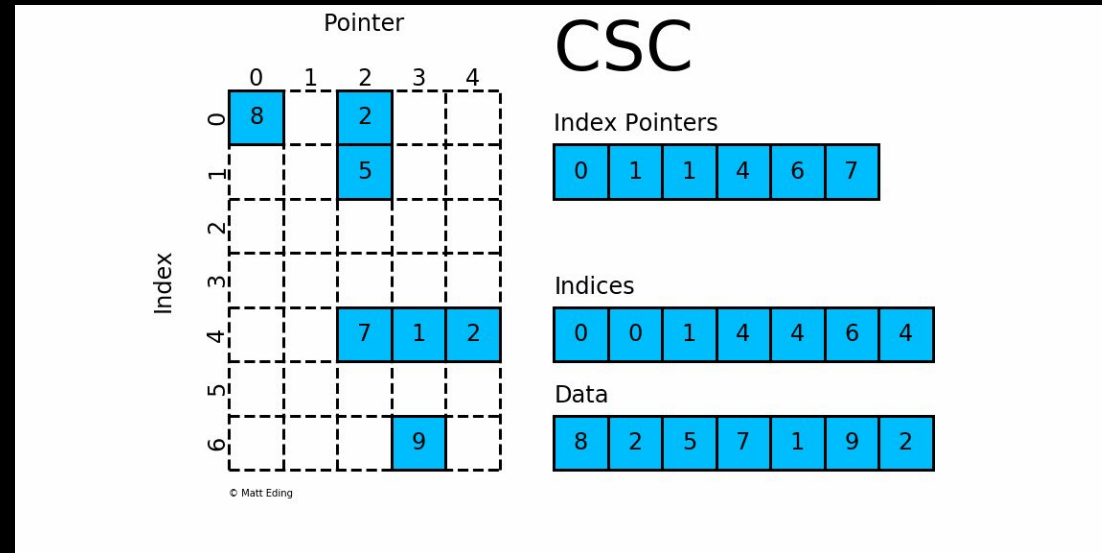
Implementazione base fornita dall'autore.
Caratterizzata da accessi casuali alla memoria e scarsa efficienza nella gestione dei grafi sparsi di grandi dimensioni.
Uso limitante della struttura CSC.

Versione MPI Originale

Primo approccio distribuito. Limitato da una comunicazione sincrona frequente, da una struttura dati (CSC) per il partizionamento dei carichi, da una centralizzazione del lavoro computazionale sul MASTER.

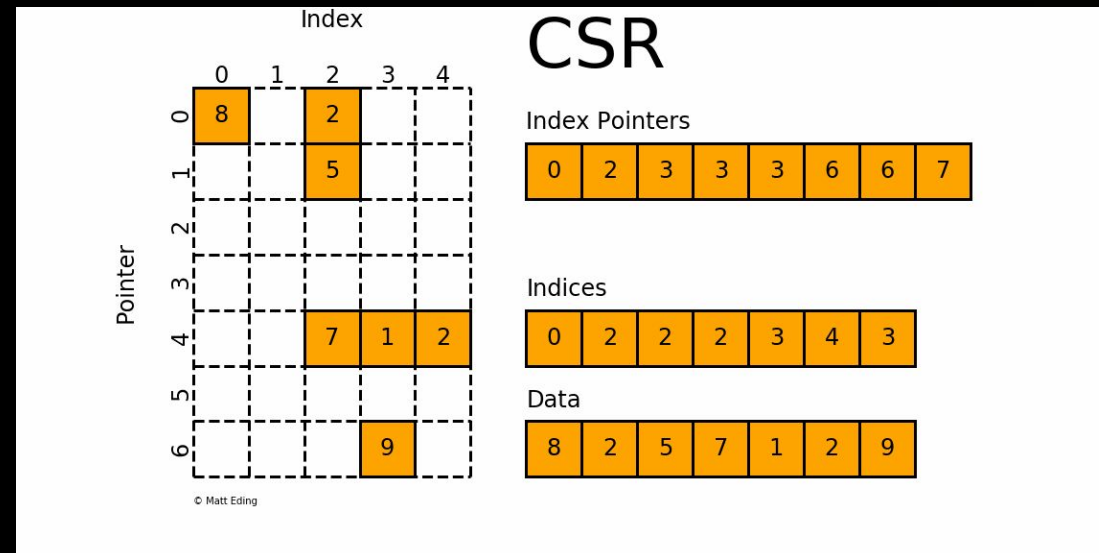
CSC (Compressed Sparse Column)

- memorizza i dati per colonna → maggiore occupazione di memoria e **poco utilizzo** della **cache**
- ogni processo/thread prende un **sottoinsieme di colonne** (nodi sorgenti)
- ogni sorgente j condivide il suo **valore di pagerank** con le sue destinazioni i
- **MPI_Reduce** sul MASTER per sommare i contributi parziali dei WORKER
- **race condition** da gestire in caso di multithreading, quindi maggiore overhead



Ottimizzazioni MPI: CSR (Compressed Sparse Row)

- memorizza i dati per righe → minor occupazione di memoria e **massimo utilizzo** della **cache**
- ogni processo/thread possiede un **sottoinsieme di righe** (nodi destinazione)
- ogni destinazione i riceve i contributi il **valore del pagerank** dalle sue sorgenti j
- distribuzione del calcolo su N processi → **elimina collo di bottiglia sul MASTER**
- **MPI_Allgatherv** per condividere il vettore del pagerank



Ottimizzazioni MPI: Efficientamento memoria

```
pr_mpi.c

...
double * damp1 = (double*)malloc(NODES*sizeof(double));
double * damp2 = (double*)malloc(NODES*sizeof(double));
double * diff = (double*)calloc(NODES, sizeof(double));

for(i=0; i<NODES;i++) {
    damp1[i] = 0.85;
    damp2[i] = 0.15 / NODES;
}
...
```

snappify.com

uso **INEFFICIENTE** della memoria
allocando spazio non necessario e
causando elevato traffico.

Risparmio di memoria con meno scritture intermedio in
RAM e maggiore velocità di esecuzione.

Unica struttura di metadati per migliorare la località della
cache, accessi più rapidi meno cache miss.

```
pr_mpi.c

...
// Costanti scalari per il Damping
const double DAMP1 = DAMPING;

int *metadata_buffer = (int*)calloc((NODES + 1) + NODES, sizeof(int));
int *rowptr          = metadata_buffer;                // Punta all'inizio del blocco
int *out_degree      = metadata_buffer + (NODES + 1); // Sfasato subito dopo rowptr
...
```

snappify.com

Ottimizzazioni MPI: Rimozione Bottleneck

```
pr_mpi.c

do {
  ...
  // Master applies damping, dangling nodes redistribution, and computes convergence
  if (rank == MASTER) {
    double dangling_mass = pagerank_compute_dangling_mass(NODES, prold, readsum);

    for(i=0; i<NODES; i++) {
      prnew[i] = prnew[i] * damp1[i] + damp2[i] + ((dangling_mass * damp1[i]) / NODES);
    }

    norm_sq = 0.0;
    for(i=0; i<NODES; i++) { // for norm calculation
      diff[i] = prnew[i] - prold[i];
      norm_sq += diff[i] * diff[i];
      prold[i] = prnew[i];
    }

    norm = sqrt(norm_sq);
  }
  ...
} while (norm > ERROR);
```

MASTER punto cruciale del calcolo!

Distribuzione del calcolo su N processi
(WORKERS + MASTER)

```
pr_mpi_opt.c

do {
  norm_sq_local = 0.0;
  dm_local = 0.0;

  // 1. Calcolo del fattore di redistribuzione basato sulla dm_global dell'iterazione PRECEDENTE
  double redistribution = dm_global * DAMP1 / NODES;

  // 2. Moltiplicazione Matrice-Vettore (SpMV) locale su formato CSR
  csr_spmv_range(rec_val, rec_colind, rowptr, prold, prnew,
    global_row_start, global_row_end, displs[rank]);

  // 3. Applicazione locale del Damping e calcolo locale del quadrato dell'errore
  pagerank_update_and_norm_range(prnew, prold,
    global_row_start, global_row_end,
    redistribution, DAMPING, NODES, &norm_sq_local);

  // 4. Pre-calcolo della Dangling Mass locale guardando avanti al vettore appena calcolato (prnew)
  dm_local = pagerank_compute_dangling_mass_range(prnew, out_degree,
    global_row_start, global_row_end);

  ...
} while (norm > ERROR);
```

Ottimizzazioni MPI: Coalescenza della comunicazione

```
pr_mpi.c

do {
...
    // Distribute PageRank vector portions
    MPI_Scatterv(prold, pcols, displs_pr, MPI_DOUBLE, rec_pr, pcols[rank], MPI_DOUBLE, 0, MPI_COMM_WORLD);

    // Sparse matrix-vector multiplication: sum = A * rec_pr
    // Only iterate over non-zero elements we actually have
    ...

    // Reduce partial results to master
    MPI_Reduce(sum, prnew, NODES, MPI_DOUBLE, MPI_SUM, MASTER, MPI_COMM_WORLD);

    // Master applies damping, dangling nodes redistribution, and computes convergence
    if (rank == MASTER) {
        ...
    }

    // Broadcast norm to all processes for loop continuation decision
    MPI_Bcast(&norm, 1, MPI_DOUBLE, MASTER, MPI_COMM_WORLD);
    ...
} while (norm > ERROR);
```

uso **MASSIVO** della comunicazione sul processo **MASTER**.

Comunicazione **RIDOTTA** con accorpamento di messaggi.

```
pr_mpi_opt.c

do {
...
    // =====
    // FUSIONE DELLE COMUNICAZIONI (Message Coalescing)
    // =====
    double local_data[2] = {dm_local, norm_sq_local};
    double global_data[2] = {0.0, 0.0};

    MPI_Allreduce(local_data, global_data, 2, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

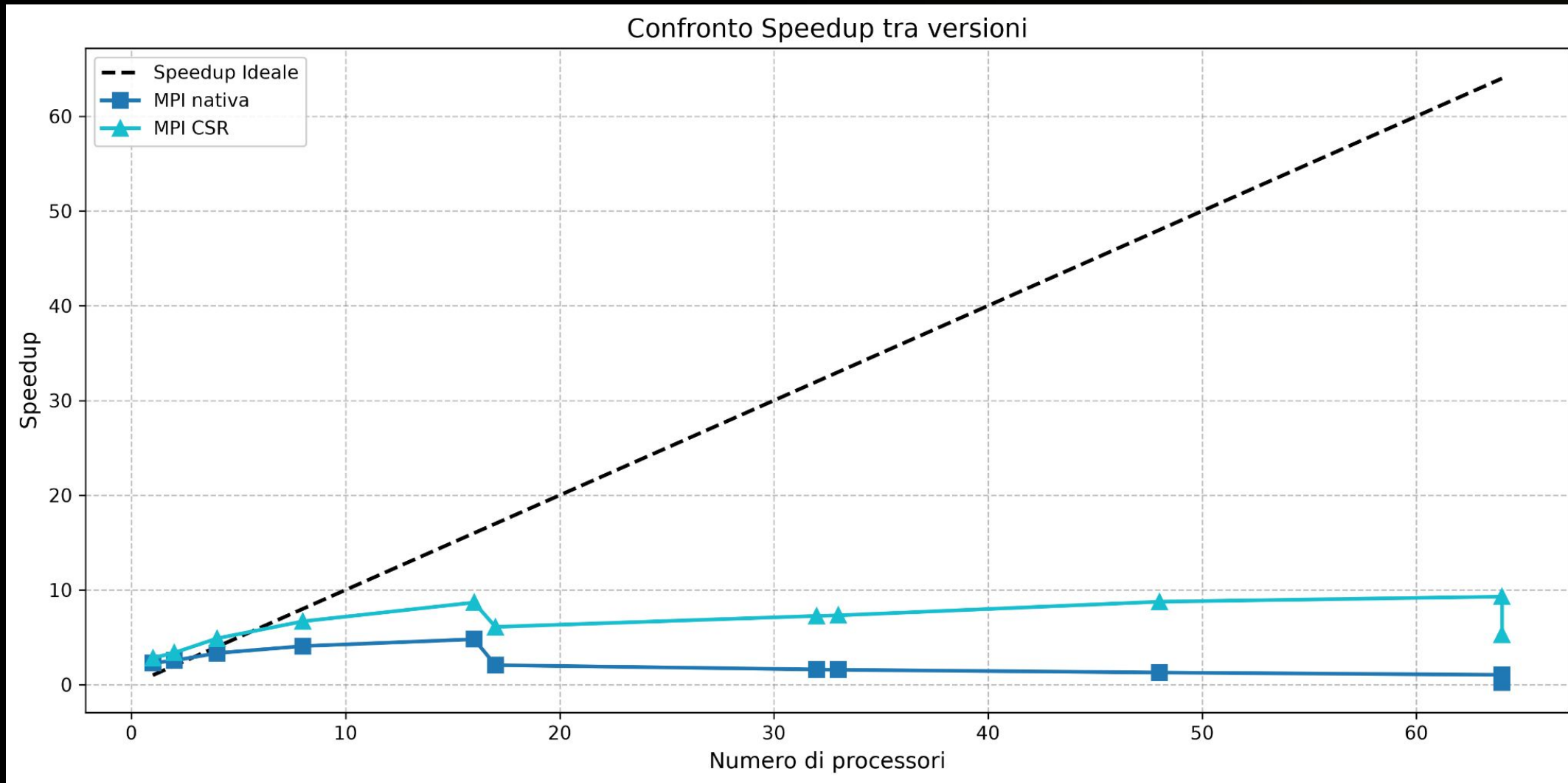
    dm_global = global_data[0];
    norm      = sqrt(global_data[1]);
    // =====

    // 5. Condivisione globale e sincrona dei blocchi di vettori aggiornati via Allgather
    MPI_Allgather(prnew + global_row_start, rec_row, MPI_DOUBLE,
                  prold, prows, displs_pr, MPI_DOUBLE,
                  MPI_COMM_WORLD);
    ...
} while (norm > ERROR);
```

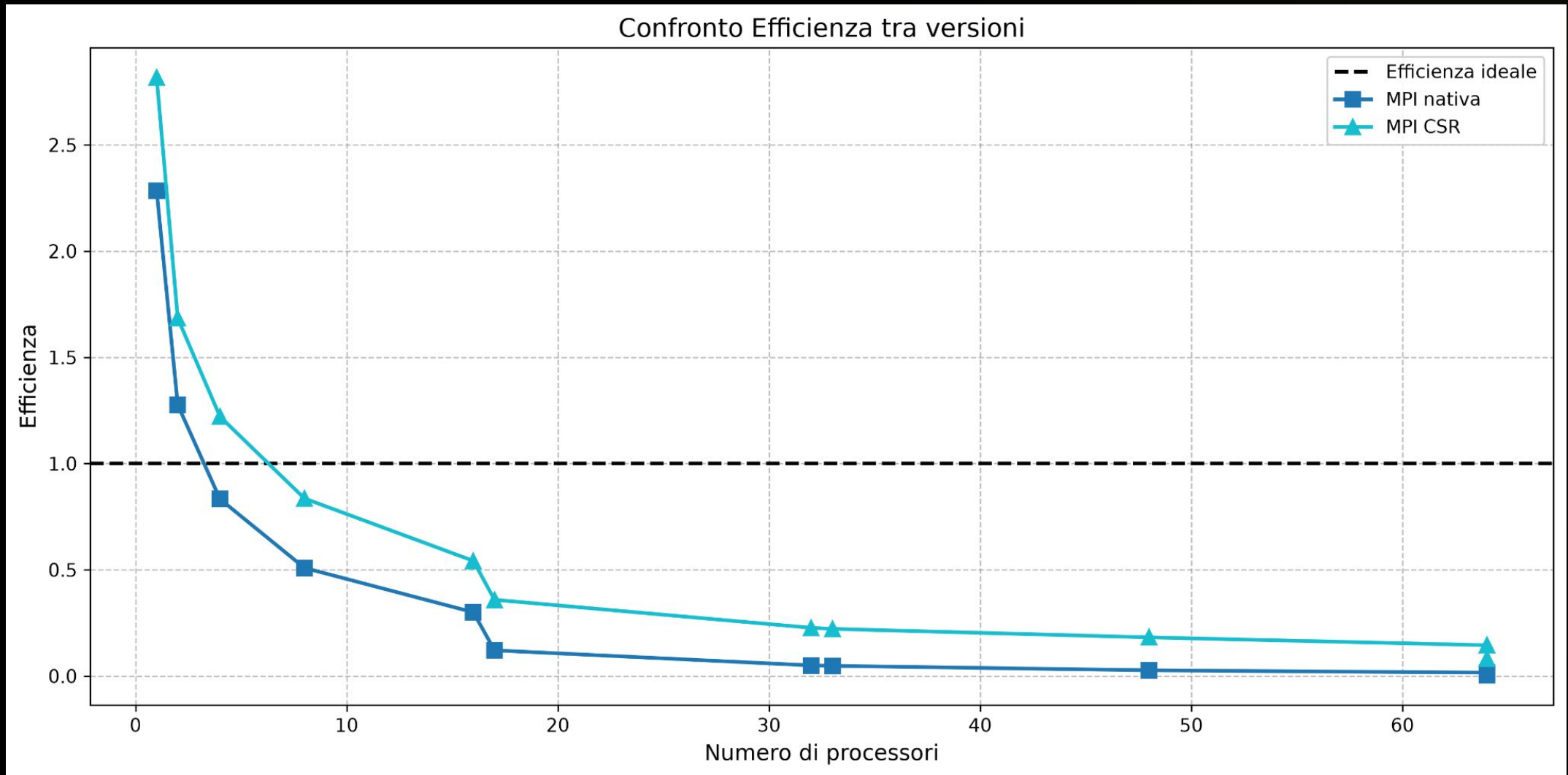
Analisi Performance

Nodi	Processori	Processi	MPI nativa			MPI ottimizzato		
			Tempo (s)	Speedup	Efficienza	Tempo (s)	Speedup	Efficienza
1	1	1	8,9626	2,2840	2,284	7,2666	2,8171	2,817
1	2	2	8,0207	2,5522	1,276	6,0778	3,3681	1,684
1	4	4	6,1440	3,3318	0,833	4,1908	4,8846	1,221
1	8	8	5,0277	4,0715	0,509	3,0625	6,6842	0,836
1	16	16	4,2696	4,7945	0,300	2,3601	8,6736	0,542
2	17	17	9,9347	2,0605	0,121	3,3568	6,0982	0,359
2	32	32	12,7495	1,6056	0,050	2,8205	7,2578	0,227
3	33	33	13,0303	1,5710	0,048	2,7989	7,3138	0,222
3	48	48	15,9715	1,2817	0,027	2,3377	8,7567	0,182
4	64	64	19,7529	1,0363	0,016	2,2029	9,2925	0,145
4	64	128	83,2452	0,2459	0,004	3,8613	5,3015	0,083

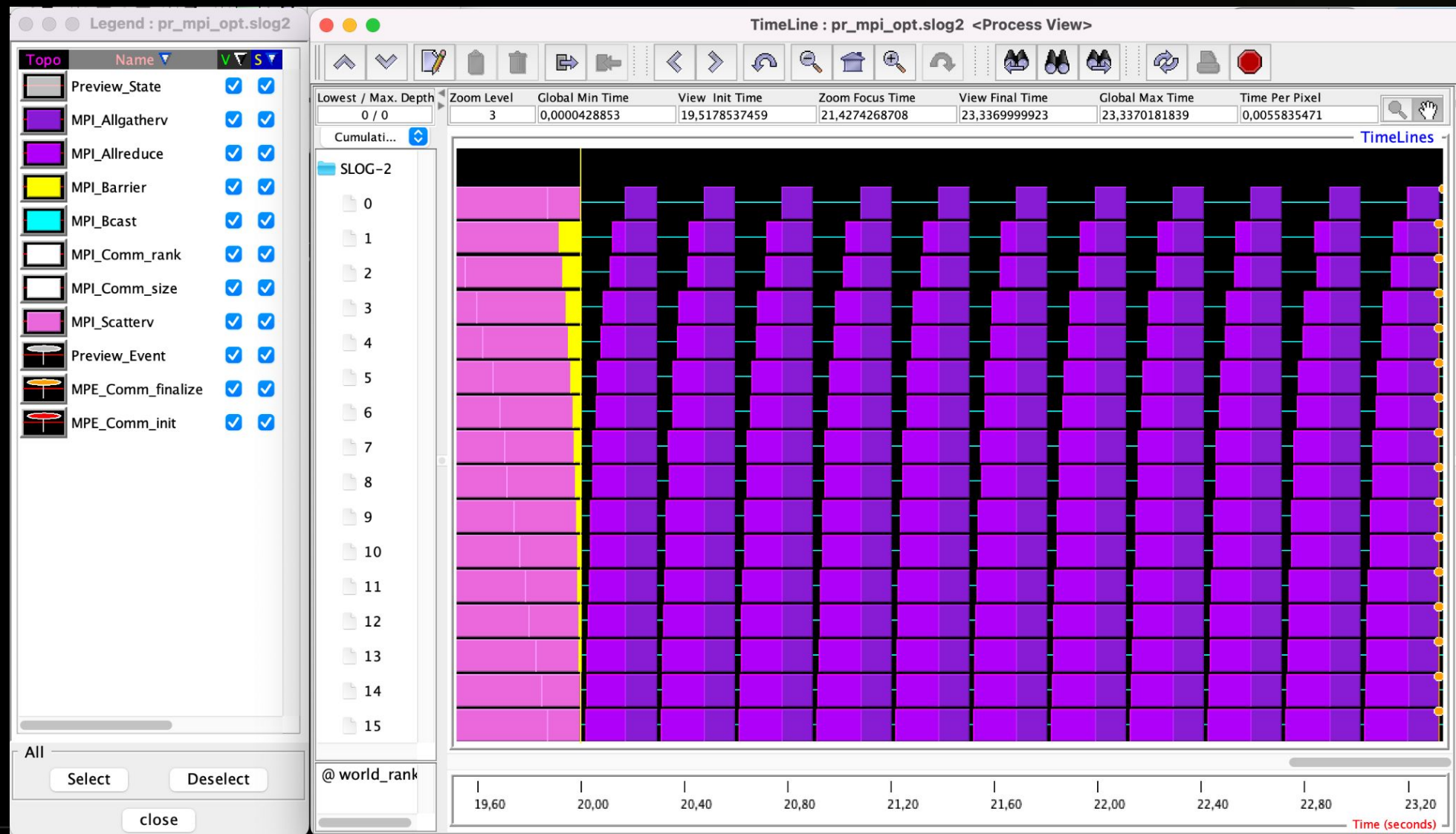
Analisi Performance: Speedup



Analisi Performance: Efficienza



Analisi delle comunicazioni



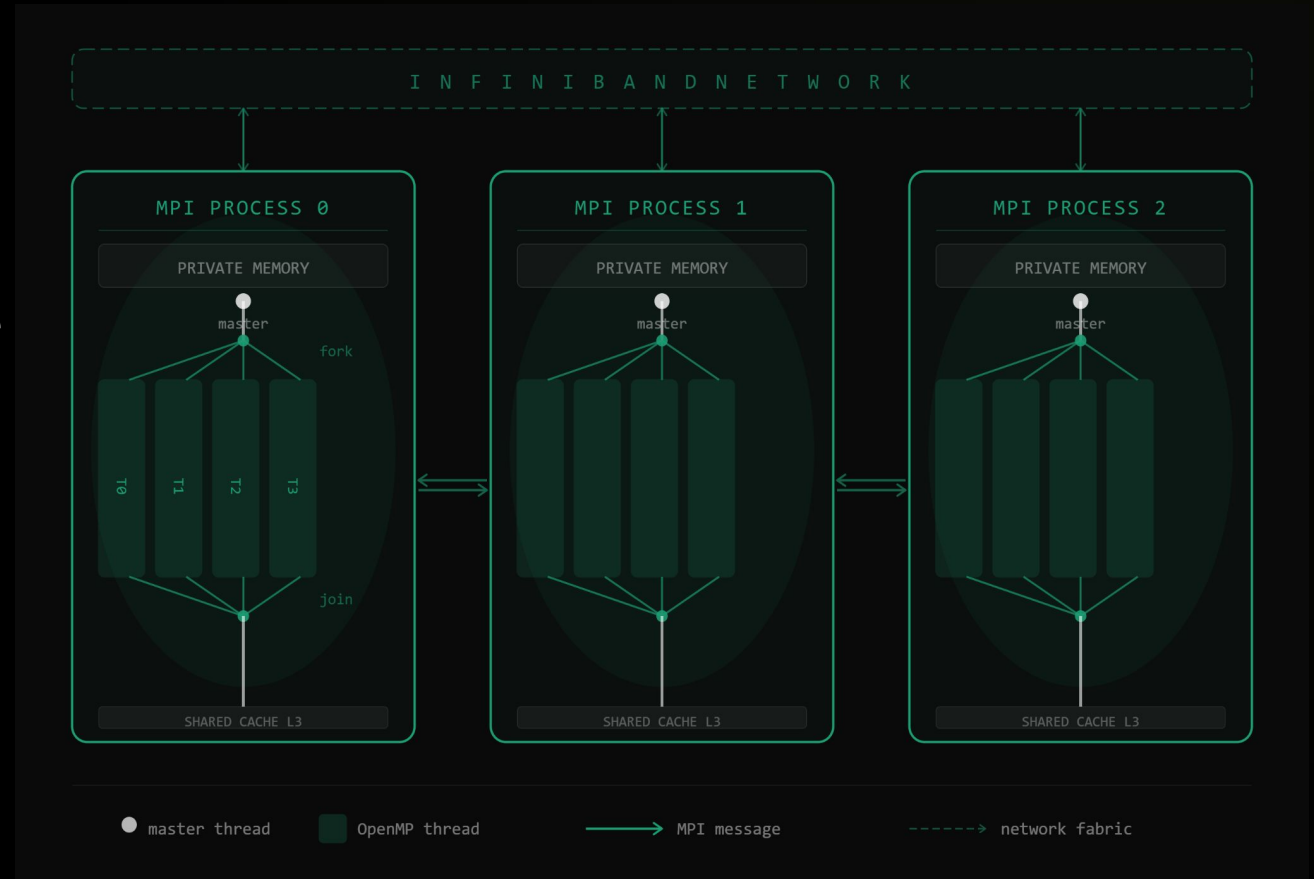


Il Paradigma Ibrido

Massimizzare l'utilizzo delle risorse computazionali del nodo.

Sinergia MPI + OpenMP

- Riduzione processi MPI → minore comunicazione
- *Sfruttamento memoria condivisa* all'interno del nodo → ottimizzazione uso cache e memoria
- Migliore scalabilità



MPI + OpenMP base: computazione

```
pr_hybrid.c

#pragma omp parallel
{
    int r, j;
    do {
        // Fase A: SpMV parallela su righe CSR (Nessun conflitto di scrittura)
        #pragma omp for reduction(+:norm_sq_local, dm_local) schedule(guided)
        for (r = global_row_start; r < global_row_end; r++) {
            int start_idx = rowptr[r] - displs[rank];
            int end_idx = rowptr[r + 1] - displs[rank];
            double row_accum = 0.0;

            for (j = start_idx; j < end_idx; j++) {
                row_accum += rec_val[j] * prold[rec_colind[j]];
            }

            prnew[r] = row_accum * DAMP1 + DAMP2 + (dm_global * DAMP1 / NODES);

            double diff = prnew[r] - prold[r];
            norm_sq_local += diff * diff;

            if (out_degree[r] == 0) {
                dm_local += prnew[r];
            }
        }
        ...
    } while (norm > ERROR);
}
```

snappify.com

→ Parallelismo a grana grossa

→ Fusione delle operazioni in un unico ciclo

parallelo: SpMV, dangling mass e norma residua

→ drastica riduzione del numero di
sincronizzazioni globali e di comunicazioni.

→ Calcolo **"in-place"** → nessun **overhead** durante il
calcolo

MPI + OpenMP base: comunicazione

```
pr_hybrid.c

#pragma omp parallel
{
    int r, j;
    do {
        ...
        // Fase C: Sincronizzazione di Rete Inter-Nodo (Solo il thread Master OpenMP comunica)
        #pragma omp master
        {
            // Message Coalescing
            double local_data[2] = {dm_local, norm_sq_local};
            double global_data[2] = {0.0, 0.0};

            MPI_Allreduce(local_data, global_data, 2, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

            dm_global = global_data[0];
            norm      = sqrt(global_data[1]);

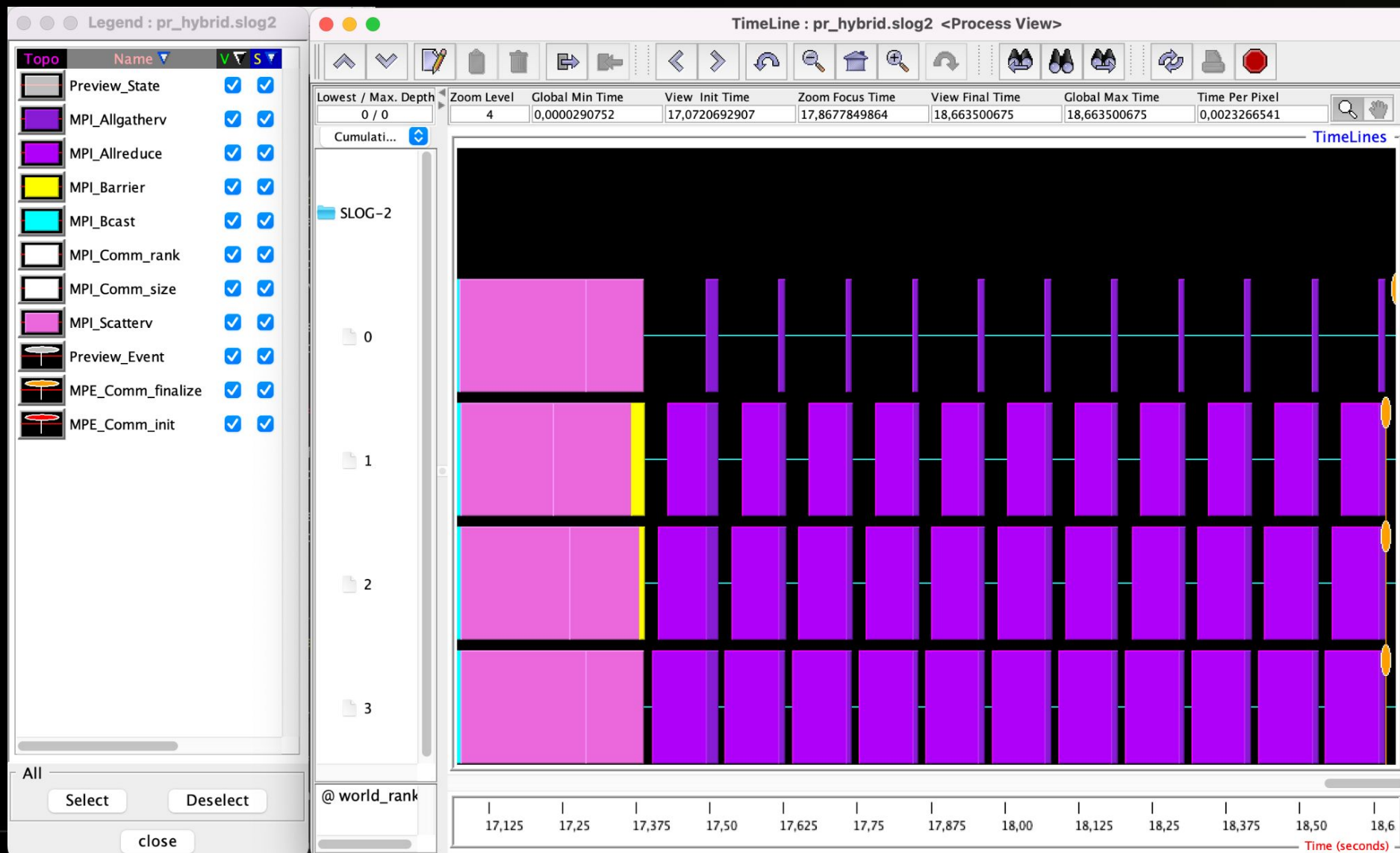
            // Allgather: Scambio leggero dei pezzi di vettore aggiornati
            MPI_Allgather(prnew + global_row_start, rec_row, MPI_DOUBLE,
                          prold, prows, displs_pr, MPI_DOUBLE,
                          MPI_COMM_WORLD);

            dm_local = 0.0;
            norm_sq_local = 0.0;
            iteration_count++;
        }
        #pragma omp barrier
    } while (norm > ERROR);
}
```

snappify.com

Comunicazione **MPI** confinata sul **thread MASTER** →
MPI_THREAD_FUNNELED

Analisi delle comunicazioni



MPI + OpenMP ottimizzato

- Adozione del **First-Touch** → miglioramento della località dei dati e riduzione degli accessi alla memoria remota (NUMA)
- **Scheduling Statico** coerente al First-Touch
- Uso **istruzioni vettoriali**
- Rimozione barriere implicite con **nowait**
- **Riduzione manuale** con **atomic**

```
pr_hybrid_opt.c

// --- OTTIMIZZAZIONE 1: NUMA FIRST-TOUCH ---
// Alloca fisicamente le pagine di memoria RAM
// in modo distribuito sui socket
#pragma omp parallel
{
    int i;
    #pragma omp for schedule(static)
    for (i = 0; i < NODES; i++) {
        prnew[i] = 0.0;
        prold[i] = 1.0 / NODES;
    }
}
```

snappify.com

```
pr_hybrid_opt.c

do {
    double local_dm = 0.0;
    double local_norm = 0.0;

    #pragma omp for schedule(static) nowait
    for (r = global_row_start; r < global_row_end; r++) {
        int start_idx = rowptr[r] - displs[rank];
        int end_idx   = rowptr[r + 1] - displs[rank];
        double row_accum = 0.0;

        // SIMD vectorization without reduction clause
        #pragma omp simd
        for (j = start_idx; j < end_idx; j++) {
            row_accum += rec_val[j] * prold[rec_colind[j]];
        }

        prnew[r] = row_accum * DAMP1 + DAMP2 + (dm_global * DAMP1 / NODES);

        double diff = prnew[r] - prold[r];
        local_norm += diff * diff;
        if(out_degree[r] == 0) local_dm += prnew[r];
    }

    // Manual reduction to avoid false sharing
    #pragma omp atomic
    norm_sq_local += local_norm;

    #pragma omp atomic
    dm_local += local_dm;

    #pragma omp barrier // Ensure all threads complete before master
    ...
} while ( norm > ERROR );
```

snappify.com

MPI + OpenMP ottimizzato: compilazione

```
$ mpicc -O3 -fopenmp <SOURCES> -o <OUTPUT> -Ilibraries -lm
```

-O3 → massimo livello di ottimizzazione del compilatore.



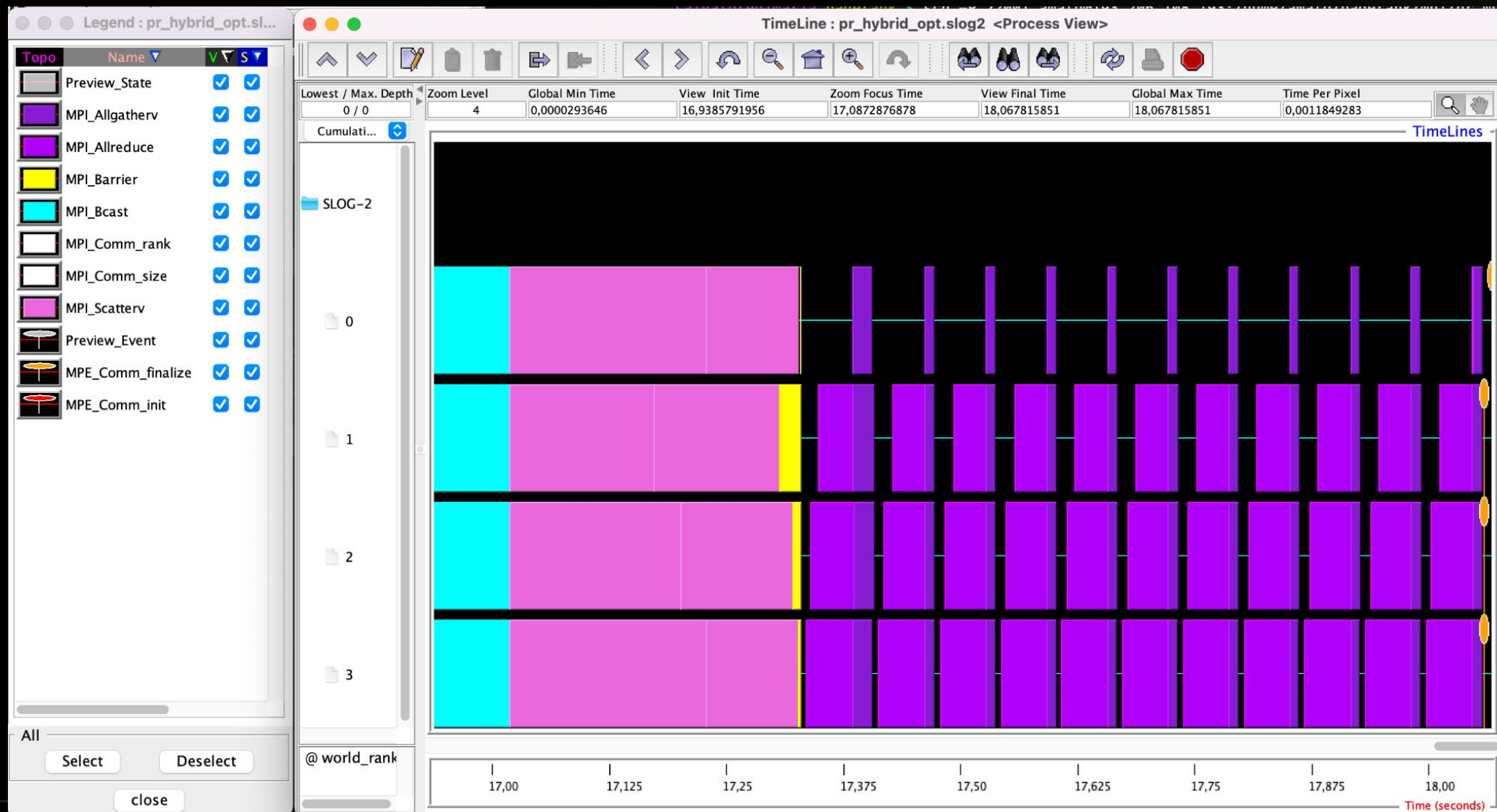
Function inlining

Loop unrolling e ottimizzazione dei cicli

Auto-vectorization (SIMD)

Eliminazione del codice ridondante e propagazione delle costanti

Analisi delle comunicazioni



Analisi Performance

--map-by node → distribuzione uniforme dei processi MPI sui nodi del cluster, favorendo il bilanciamento del carico.

Utilizzo della rete InfiniBand nativa → riduzione della latenza e aumento della banda nelle comunicazioni MPI tra nodi.

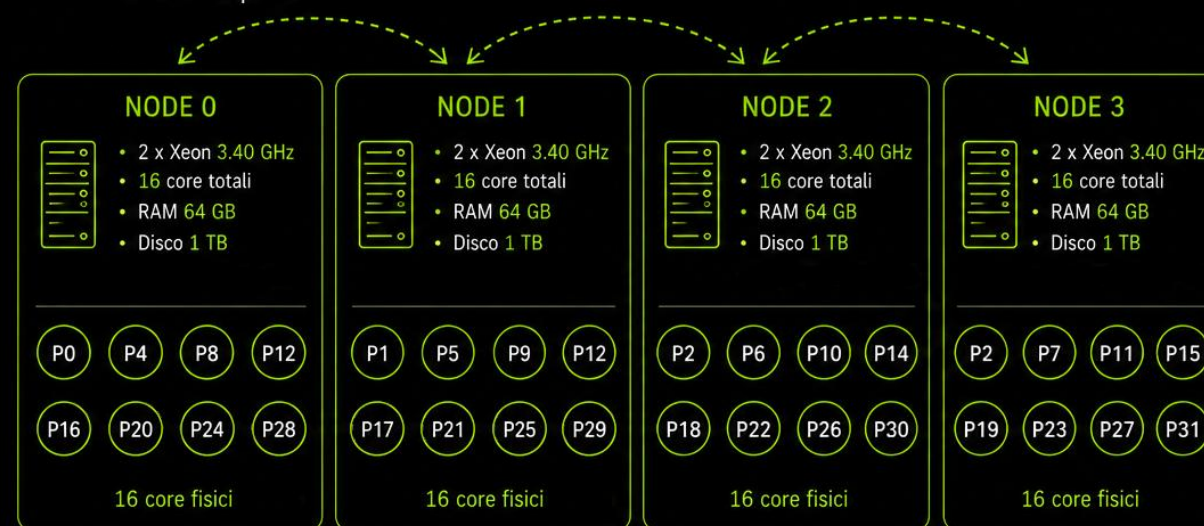
Massimo utilizzo delle risorse hardware → configurazioni scelte per occupare **sempre tutti i core disponibili** del cluster.

Scalabilità ibrida → confronto di diverse configurazioni MPI+OpenMP mantenendo costante il numero di processori utilizzati (es. più processi e meno thread, oppure meno processi e più thread).



ROUND ROBIN (--map-by nodes):

i processi MPI vengono distribuiti in modo ciclico (round robin) tra i nodi disponibili.

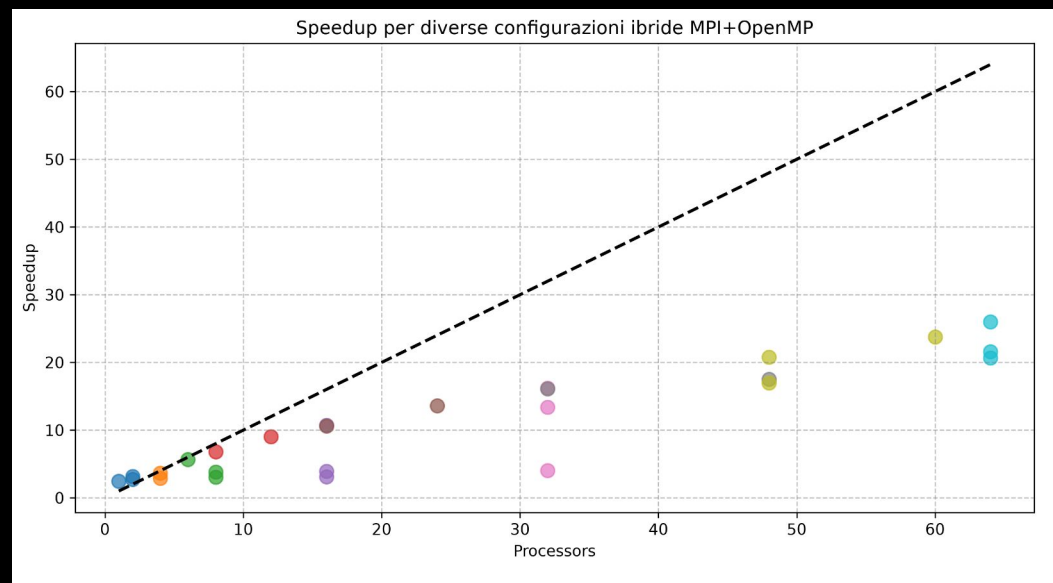


Analisi Performance

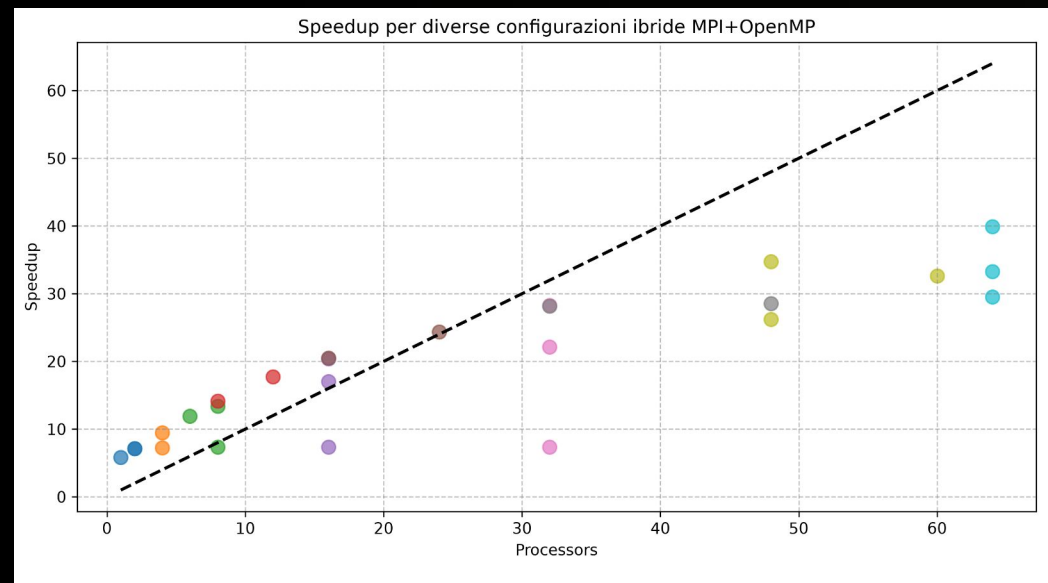
					MPI + OpenMP base			MPI + OpenMP ottimizzato		
Processori	Configurazioni (Proc./Thread)				Tempo (s)	Speedup	Efficienza	Tempo (s)	Speedup	Efficienza
1	1/1				8,3068	2,4643	2,464	3,5084	5,8347	5,835
2	1/2	2/1			6,5085	3,1452	1,573	2,8601	7,1573	3,579
4	1/4	2/2			5,5655	3,6781	0,920	2,1643	9,4583	2,365
6	3/2	2/3			3,6044	5,6793	0,947	1,7205	11,8980	1,983
8	1/8	2/4	4/2		3,0005	9,0123	0,853	1,4458	14,1586	1,777
12	3/4				2,2714	6,8224	0,751	1,1533	17,7495	1,479
16	1/16	2/8	4/4	8/2	1,9100	10,7175	0,670	0,9989	20,4930	1,281
24	3/8				1,5059	13,5935	0,566	0,8399	24,3725	1,016
32	2/16	4/8	8/4	16/2	1,2615	16,2271	0,507	0,7236	28,2898	0,884
48	16/3	8/6	3/16		0,9845	20,7928	0,433	0,5893	34,7370	0,724
60	6/10				0,8603	23,7946	0,397	0,6273	32,6327	0,544
64	4/16	8/8	16/4		0,7879	25,9811	0,406	0,5132	39,8880	0,623

Analisi Performance: Speedup

L'ottimizzazione migliora sensibilmente lo speedup rispetto alla versione base, in particolare nella fascia media di processori, ma il divario dallo speedup ideale resta significativo oltre i 40 core a causa dell'overhead di comunicazione.



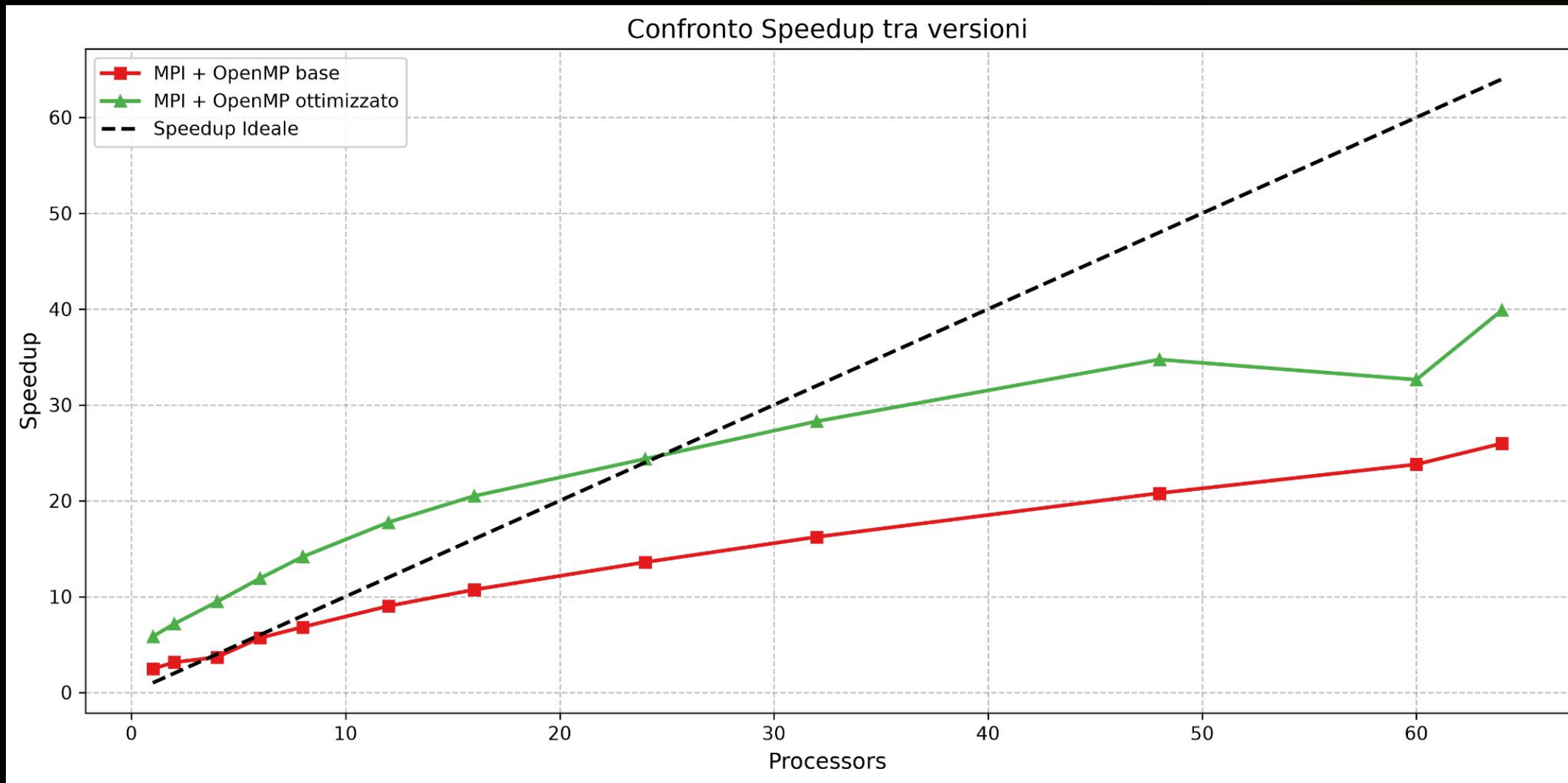
MPI + OpenMP base



MPI + OpenMP ottimizzato

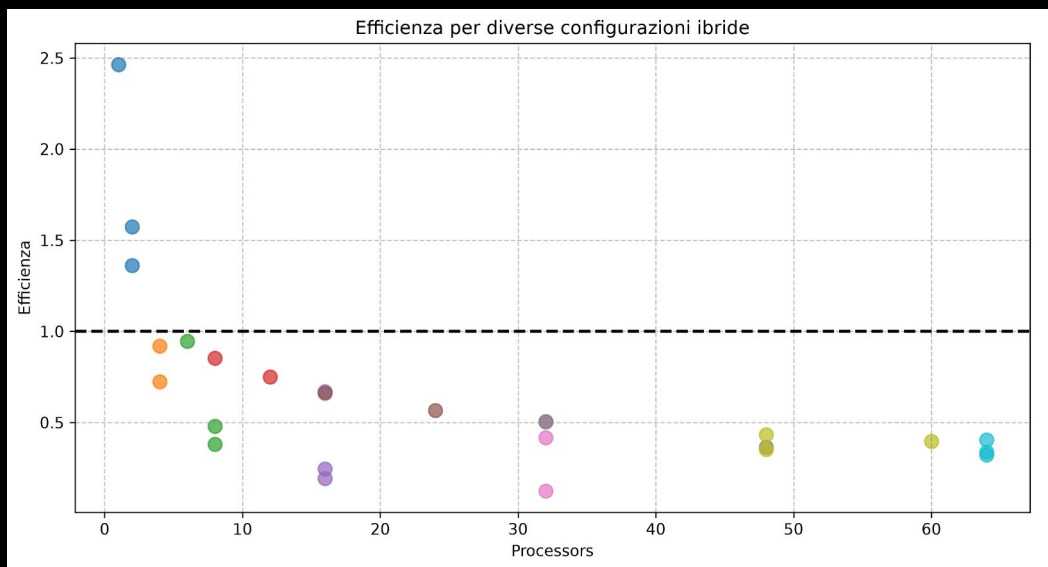
- 1 nodi, 1 MPI, 1 OMP
- 1 nodi, 1 MPI, 2 OMP
- 2 nodi, 2 MPI, 1 OMP
- 1 nodi, 1 MPI, 4 OMP
- 2 nodi, 2 MPI, 2 OMP
- 3 nodi, 3 MPI, 2 OMP
- 2 nodi, 2 MPI, 4 OMP
- 1 nodi, 1 MPI, 8 OMP
- 4 nodi, 4 MPI, 2 OMP
- 3 nodi, 3 MPI, 4 OMP
- 1 nodi, 1 MPI, 16 OMP
- 2 nodi, 2 MPI, 8 OMP
- 4 nodi, 4 MPI, 4 OMP
- 4 nodi, 8 MPI, 2 OMP
- 3 nodi, 3 MPI, 8 OMP
- 2 nodi, 2 MPI, 16 OMP
- 4 nodi, 16 MPI, 2 OMP
- 4 nodi, 4 MPI, 8 OMP
- 4 nodi, 8 MPI, 4 OMP
- 3 nodi, 3 MPI, 16 OMP
- 4 nodi, 8 MPI, 6 OMP
- 4 nodi, 16 MPI, 3 OMP
- 4 nodi, 6 MPI, 10 OMP
- 4 nodi, 4 MPI, 16 OMP
- 4 nodi, 8 MPI, 8 OMP
- 4 nodi, 16 MPI, 4 OMP
- Speedup ideale

Analisi Performance: Speedup

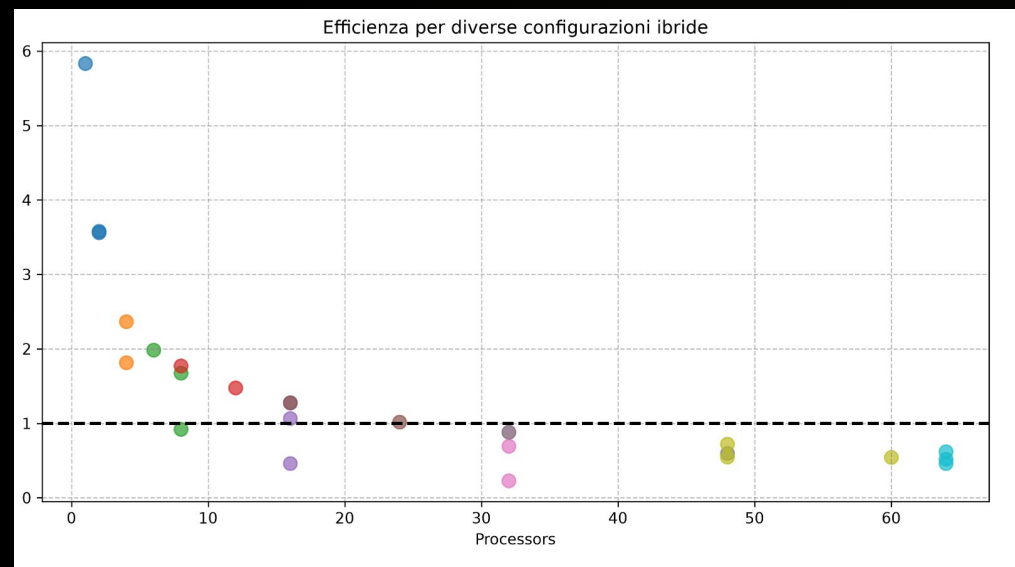


Analisi Performance: Efficienza

L'efficienza cala progressivamente all'aumentare dei processori in entrambe le versioni, ma la versione ottimizzata mostra un guadagno iniziale più marcato e mantiene valori più vicini o superiori all'ideale, per poi convergere anch'essa verso livelli bassi ad alta parallelizzazione.



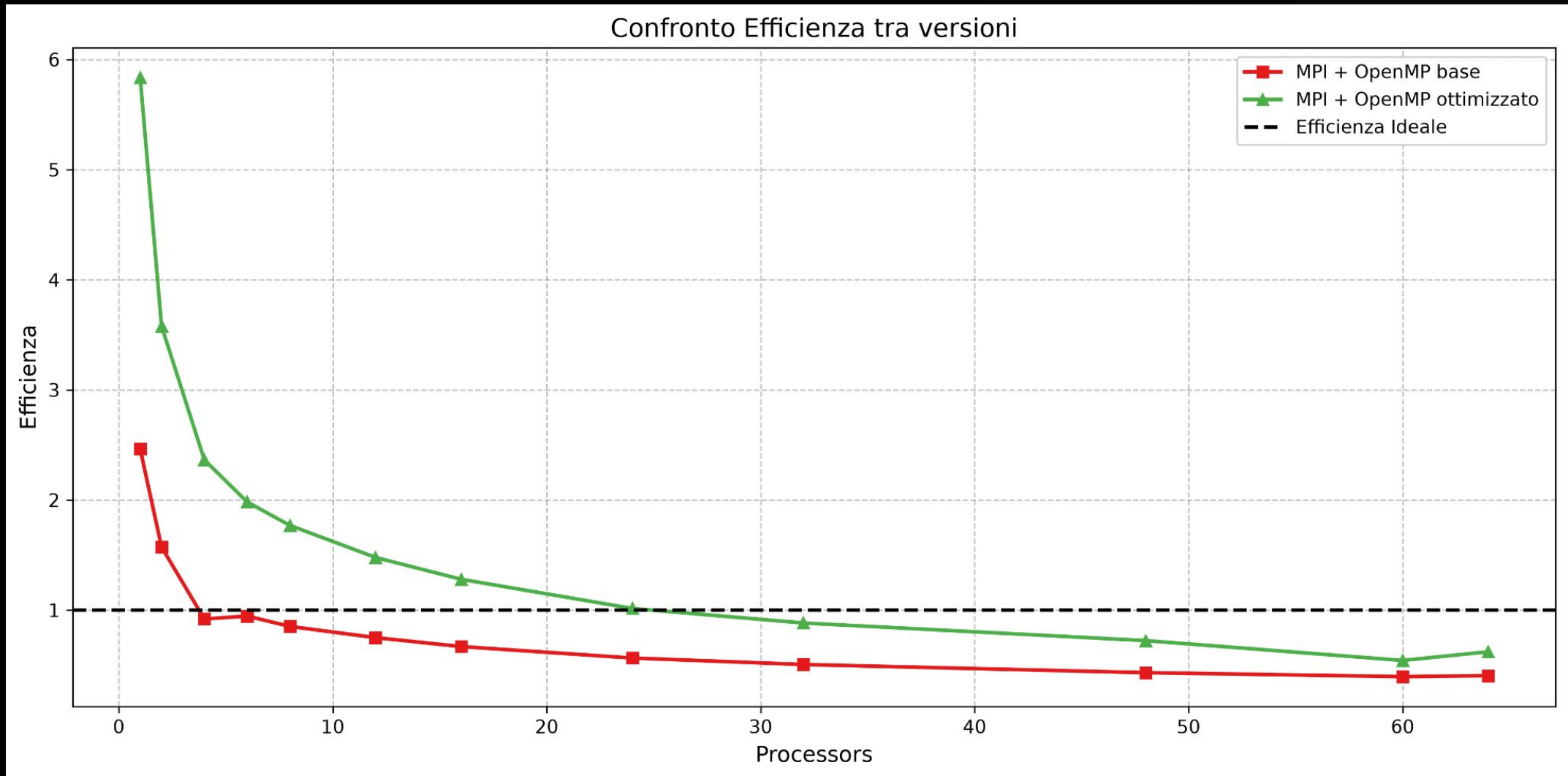
MPI + OpenMP base



MPI + OpenMP ottimizzato

- 1 nodi, 1 MPI, 1 OMP
- 1 nodi, 1 MPI, 2 OMP
- 2 nodi, 2 MPI, 1 OMP
- 1 nodi, 1 MPI, 4 OMP
- 2 nodi, 2 MPI, 2 OMP
- 3 nodi, 3 MPI, 2 OMP
- 2 nodi, 2 MPI, 4 OMP
- 1 nodi, 1 MPI, 8 OMP
- 4 nodi, 4 MPI, 2 OMP
- 3 nodi, 3 MPI, 4 OMP
- 1 nodi, 1 MPI, 16 OMP
- 2 nodi, 2 MPI, 8 OMP
- 4 nodi, 4 MPI, 4 OMP
- 4 nodi, 8 MPI, 2 OMP
- 3 nodi, 3 MPI, 8 OMP
- 2 nodi, 2 MPI, 16 OMP
- 4 nodi, 16 MPI, 2 OMP
- 4 nodi, 4 MPI, 8 OMP
- 4 nodi, 8 MPI, 4 OMP
- 3 nodi, 3 MPI, 16 OMP
- 4 nodi, 8 MPI, 6 OMP
- 4 nodi, 16 MPI, 3 OMP
- 4 nodi, 6 MPI, 10 OMP
- 4 nodi, 4 MPI, 16 OMP
- 4 nodi, 8 MPI, 8 OMP
- 4 nodi, 16 MPI, 4 OMP
- Efficienza ideale

Analisi Performance: Efficienza



Evoluzione Asincrona

Overlap calcolo e comunicazione

MPI + OpenMP **asincrono**

- **Contesto**: ogni processo MPI gestisce delle righe (destinazioni) di competenza e i relativi archi, che potremmo considerare come archi locali e remoti, e legge il vettore ***prold*** per calcolare il nuovo Pagerank.
 - **Archi locali**: puntano a destinazioni già possedute dal processo (i valori di ***prold*** sono già disponibili senza attendere la rete).
 - **Archi remoti**: puntano a destinazioni possedute da altri processi e dunque disponibili solo dopo comunicazioni via rete.
 - **Idea chiave**: sovrapporre temporalmente calcolo su archi locali (indipendente dalla comunicazione) con la ricezione asincrona del vettore pagerank aggiornato, necessario per elaborare gli archi remoti.
-

MPI + OpenMP **asincrono**: pre-processing

Rispetto alla versione precedente si è aggiunto un **vettore di accumulo per i contributi locali** che verrà usato durante le iterazioni asincrone



Overlap calcolo locale e comunicazioni dei valori remoti.

```
pr_hybrid_async.c

// Buffer per memorizzare i risultati parziali (solo archi locali)
double *partial_prnew = (double *)malloc(NODES * sizeof(double));

#pragma omp parallel
{
    int i;
    #pragma omp for schedule(static)
    for (i = 0; i < NODES; i++) {
        prnew[i] = 0.0;
        partial_prnew[i] = 0.0;
        prold[i] = 1.0 / NODES;
    }
}
```

snappify.com

MPI + OpenMP asincrono: pre-processing

```
pr_hybrid_async.c

// =====
// PRE-PROCESSING PARALLELO: DIVISIONE ARCHI LOCALI E REMOTI (GHOST NODES)
// =====
int global_row_start = displs_pr[rank];
int global_row_end   = global_row_start + rec_row;

int *loc_rowptr = (int *)calloc(rec_row + 1, sizeof(int));
int *rem_rowptr = (int *)calloc(rec_row + 1, sizeof(int));

// 1. CONTEGGIO IN PARALLELO (Nessuna dipendenza tra righe r)
int r, j;
#pragma omp parallel for schedule(static) private(j)
for (r = 0; r < rec_row; r++) {
    int start_idx = rowptr[global_row_start + r] - displs[rank];
    int end_idx   = rowptr[global_row_start + r + 1] - displs[rank];
    for (j = start_idx; j < end_idx; j++) {
        int target = rec_colind[j];
        if (target >= global_row_start && target < global_row_end) {
            loc_rowptr[r + 1]++;
        } else {
            rem_rowptr[r + 1]++;
        }
    }
}

// 2. PREFIX SUM (Deve restare sequenziale: loop-carried dependency)
for (r = 0; r < rec_row; r++) {
    loc_rowptr[r + 1] += loc_rowptr[r];
    rem_rowptr[r + 1] += rem_rowptr[r];
}

...
```

- **Conteggio parallelo** degli archi locali e dei remoti, per ogni riga.
- **Conversione conteggi** in offset di inizio per ogni riga: in questo modo riusciamo a sapere dove iniziano e dove finiscono gli archi di una riga all'interno degli array.
- Fase di **predisposizione** alla creazione di **due strutture CSR minori**, una per calcolo locale ed una per il remoto.

MPI + OpenMP asincrono: pre-processing

Popolamento parallelo delle strutture
local/remote ottenendo **due mini CSR**.



Ogni thread ripercorre le righe di propria
competenze e smista gli archi in base alla
destinazione.

```
pr_hybrid_async.c

// 3. POPOLAMENTO IN PARALLELO (Ogni thread scrive nella propria fetta contigua senza conflitti)
#pragma omp parallel for schedule(static) private(j)
for (r = 0; r < rec_row; r++) {
    int start_idx = rowptr[global_row_start + r] - displs[rank];
    int end_idx   = rowptr[global_row_start + r + 1] - displs[rank];
    for (j = start_idx; j < end_idx; j++) {
        int target = rec_colind[j];
        double weight = rec_val[j];
        if (target >= global_row_start && target < global_row_end) {
            int idx = loc_rowptr[r] + loc_offset[r]++;
            loc_colind[idx] = target;
            loc_val[idx] = weight;
        } else {
            int idx = rem_rowptr[r] + rem_offset[r]++;
            rem_colind[idx] = target;
            rem_val[idx] = weight;
        }
    }
}
...

```

snappify.com

MPI + OpenMP asincrono: computazione

Durante l'iterazione si opera sulla porzione locale senza attendere la comunicazione per poi passare immediatamente al calcolo remoto.

```
pr_hybrid_async.c

#pragma omp parallel
{
    int r, j;
    do {
        double local_dm = 0.0;
        double local_norm = 0.0;

        // operazione async
        ...

        // --- FASE 1: CALCOLO LOCALE (Mentre la rete trasmette in background) ---
        #pragma omp for schedule(static) nowait
        for (r = 0; r < rec_row; r++) {
            int global_r = global_row_start + r;
            double row_accum = 0.0;
            #pragma omp simd reduction(+:row_accum)
            for (j = loc_rowptr[r]; j < loc_rowptr[r + 1]; j++) {
                row_accum += loc_val[j] * prold[loc_colind[j]];
            }
            partial_prnew[global_r] = row_accum;
        }

        ...
    } while ( norm > ERROR );
}
```

```
pr_hybrid_async.c

#pragma omp parallel
{
    int r, j;
    do {
        ...

        // --- FASE 2: ATTESA RETE ---
        ...

        // --- FASE 3: CALCOLO REMOTO E FINALIZZAZIONE ---
        #pragma omp for schedule(static) nowait
        for (r = 0; r < rec_row; r++) {
            int global_r = global_row_start + r;
            double row_accum = partial_prnew[global_r]; // Start with local sum

            // Add remote contributions
            #pragma omp simd reduction(+:row_accum)
            for (j = rem_rowptr[r]; j < rem_rowptr[r + 1]; j++) {
                row_accum += rem_val[j] * prold[rem_colind[j]];
            }

            // Apply the formula ONCE with the complete sum
            prnew[global_r] = row_accum * DAMP1 + DAMP2 + (dm_global * DAMP1 / NODES);

            double diff = prnew[global_r] - prold[global_r];
            local_norm += diff * diff;

            // Use prold for dangling mass computation
            if(out_degree[global_r] == 0) {
                local_dm += prnew[global_r];
            }
        }

        // Manual reduction to avoid false sharing
        #pragma omp atomic
        norm_sq_local += local_norm;

        #pragma omp atomic
        dm_local += local_dm;

        #pragma omp barrier // Ensure all threads complete before master
        ...
    } while ( norm > ERROR );
}
```

MPI + OpenMP **asincrono**: comunicazione

- **Raccolta non bloccante e in place** delle porzioni remote del Pagerank: mentre la rete trasmette, i thread del processo computano la parte locale.
- Fondamentale assicurarsi che **MPI_Iallgatherv** sia terminata prima di passare alla parte remota.

Osservazione: Con un large-scale graph, il carico computazionale è tale da nascondere la latenza di comunicazione, evitando che i thread rimangano in attesa del completamento degli scambi di dati.

```
pr_hybrid_async.c

#pragma omp parallel
{
    int r, j;
    do {
        #pragma omp master
        {
            if (iteration_count > 0) {
                double *tmp = prold;
                prold = prnew;
                prnew = tmp;
            }

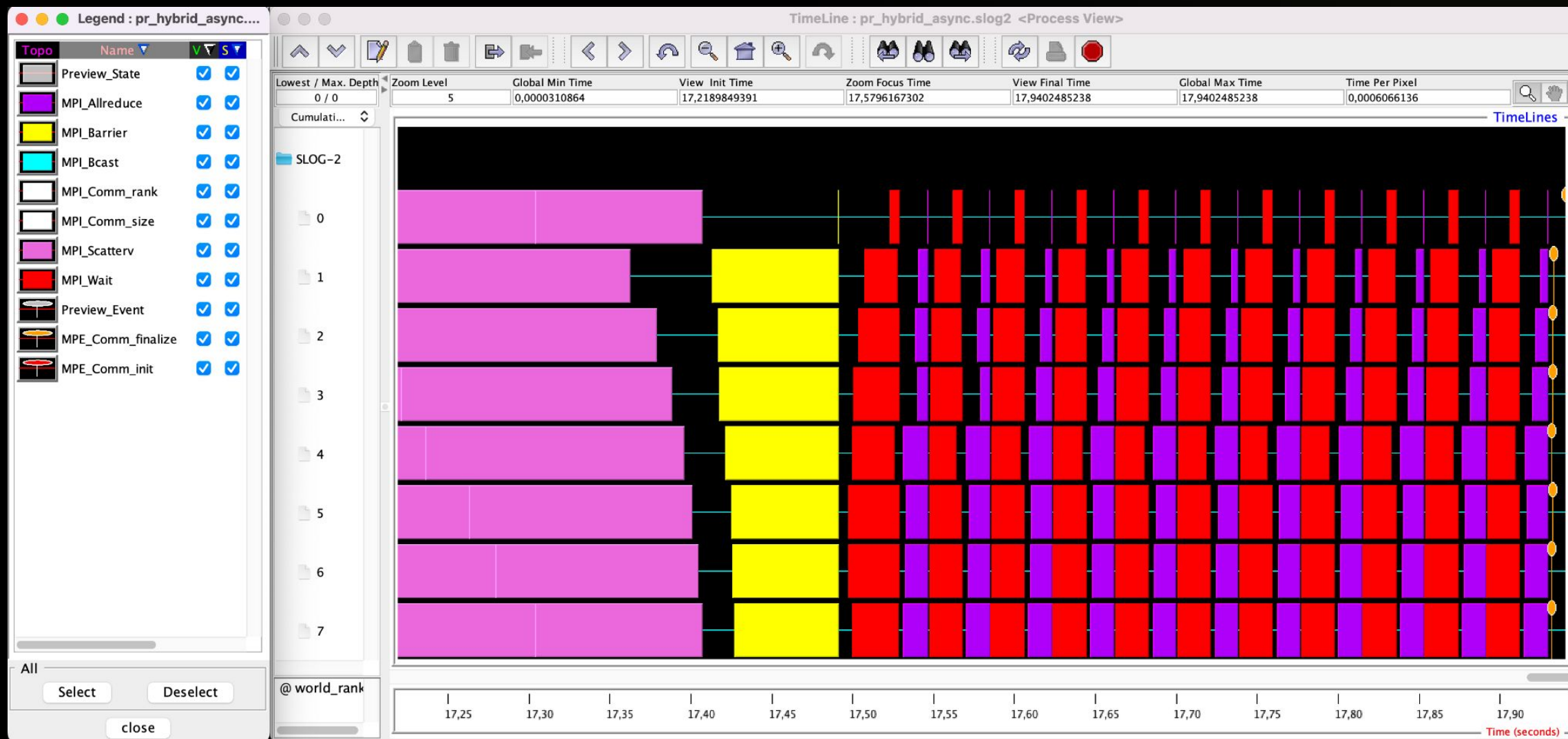
            // Uso di MPI_IN_PLACE per evitare Data Race con la Fase 1
            MPI_Iallgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL,
                           prold, prows, displs_pr, MPI_DOUBLE,
                           MPI_COMM_WORLD, &req_pr);
        }

        // Barriera post-swap per far partire i thread all'unisono
        #pragma omp barrier
        ...

        // --- FASE 2: ATTESA RETE ---
        #pragma omp master
        {
            MPI_Wait(&req_pr, MPI_STATUS_IGNORE);
        }
        #pragma omp barrier // Tutti i thread aspettano i dati
        ...
    } while ( norm > ERROR );
}
```

snappify.com

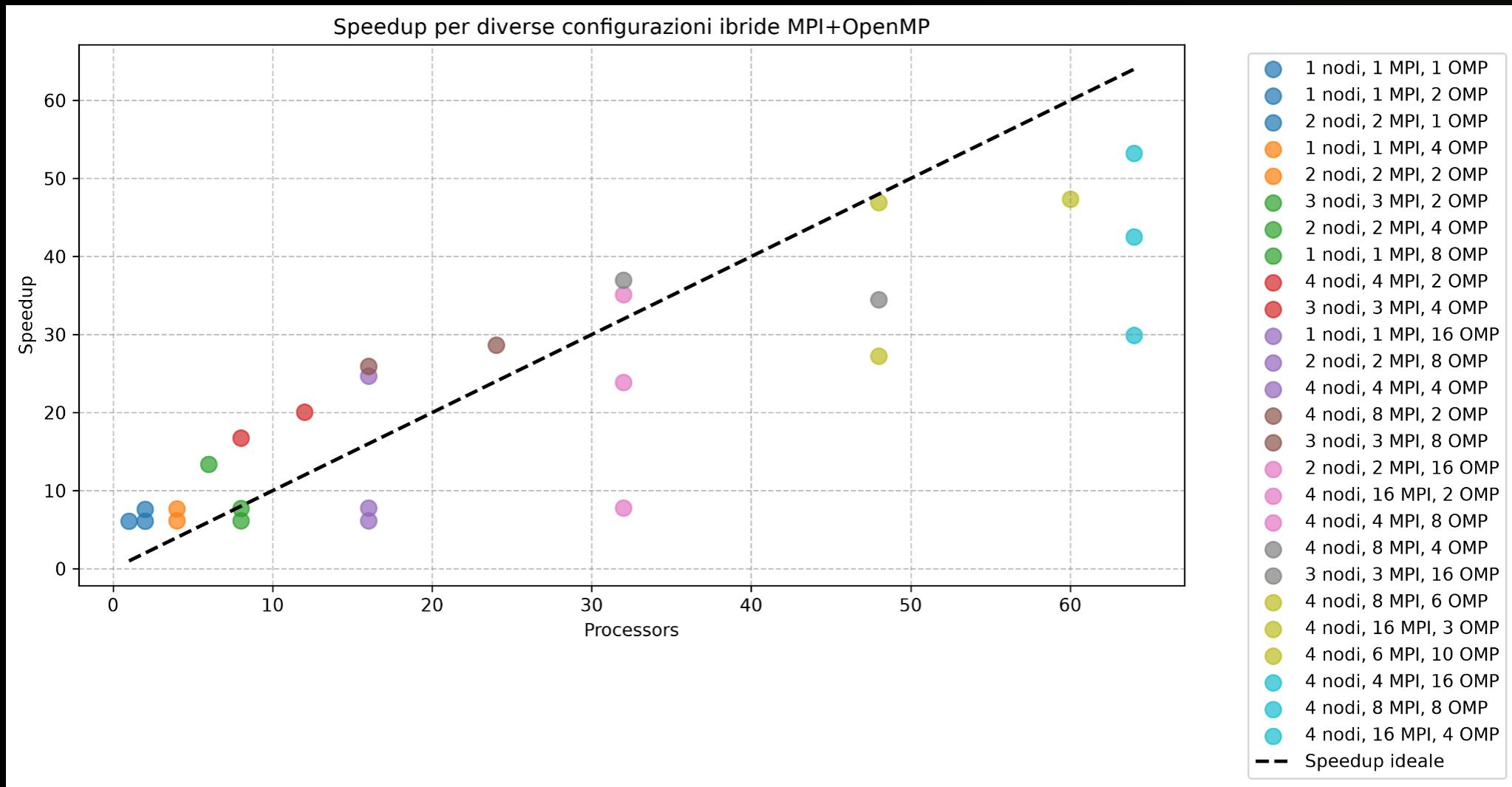
Analisi delle comunicazioni



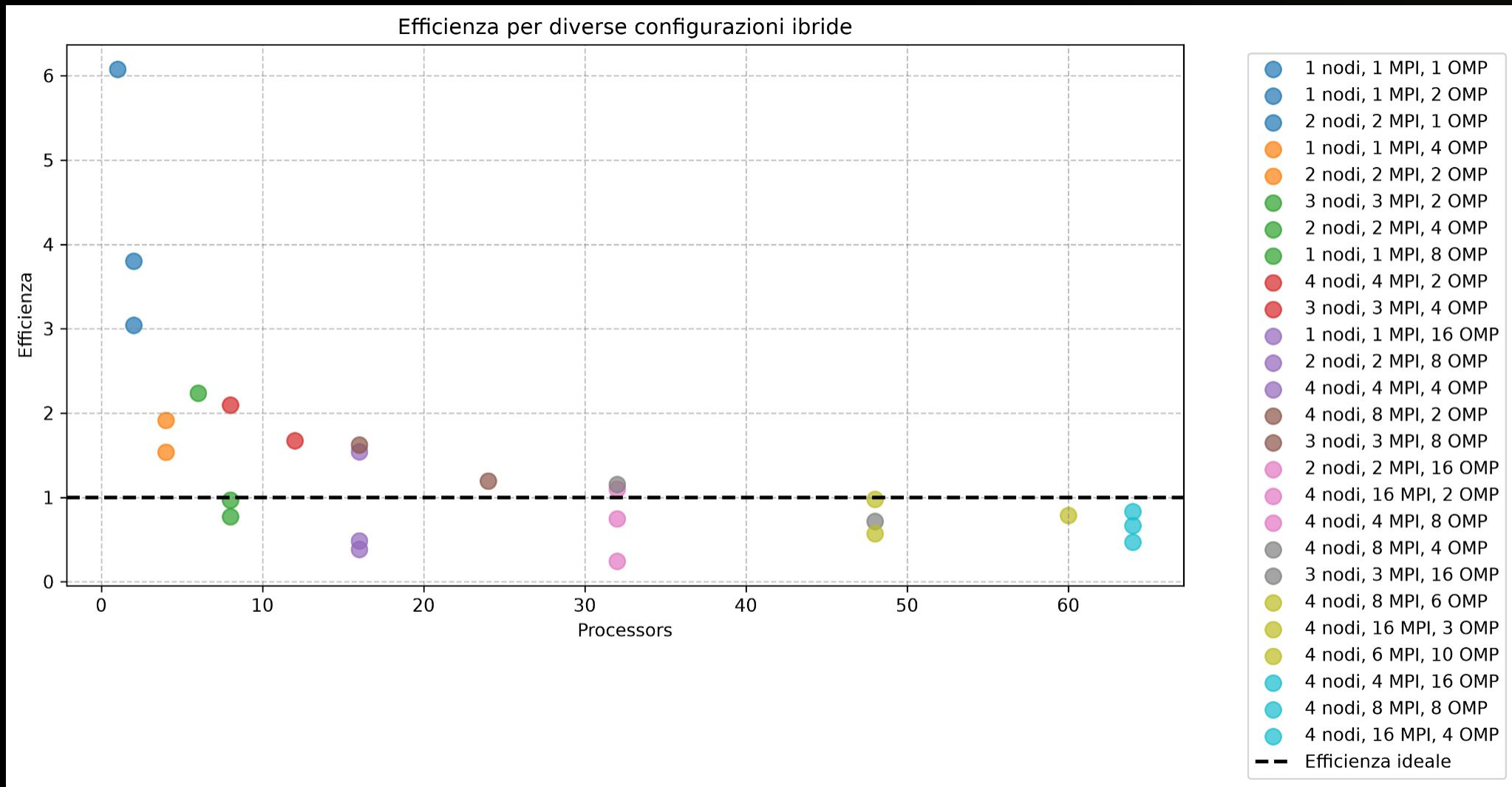
Analisi Performance

Processori	Configurazioni (Proc./Thread)				Tempo (s)	Speedup	Efficienza
1	1/1				3,3685	6,0770	6,077
2	1/2	2/1			2,6917	7,6050	3,802
4	1/4	2/2			2,6741	7,6551	1,914
6	3/2	2/3			1,5267	13,4083	2,235
8	1/8	2/4	4/2		1,2206	16,7709	2,096
12	3/4				1,0191	20,0868	1,674
16	1/16	2/8	4/4	8/2	0,7894	25,9317	1,621
24	3/8				0,7136	28,6862	1,195
32	2/16	4/8	8/4	16/2	0,5540	36,9504	1,155
48	16/3	8/6	3/16		0,4361	46,9399	0,978
60	6/10				0,4323	47,3525	0,789
64	4/16	8/8	16/4		0,3846	53,2254	0,832

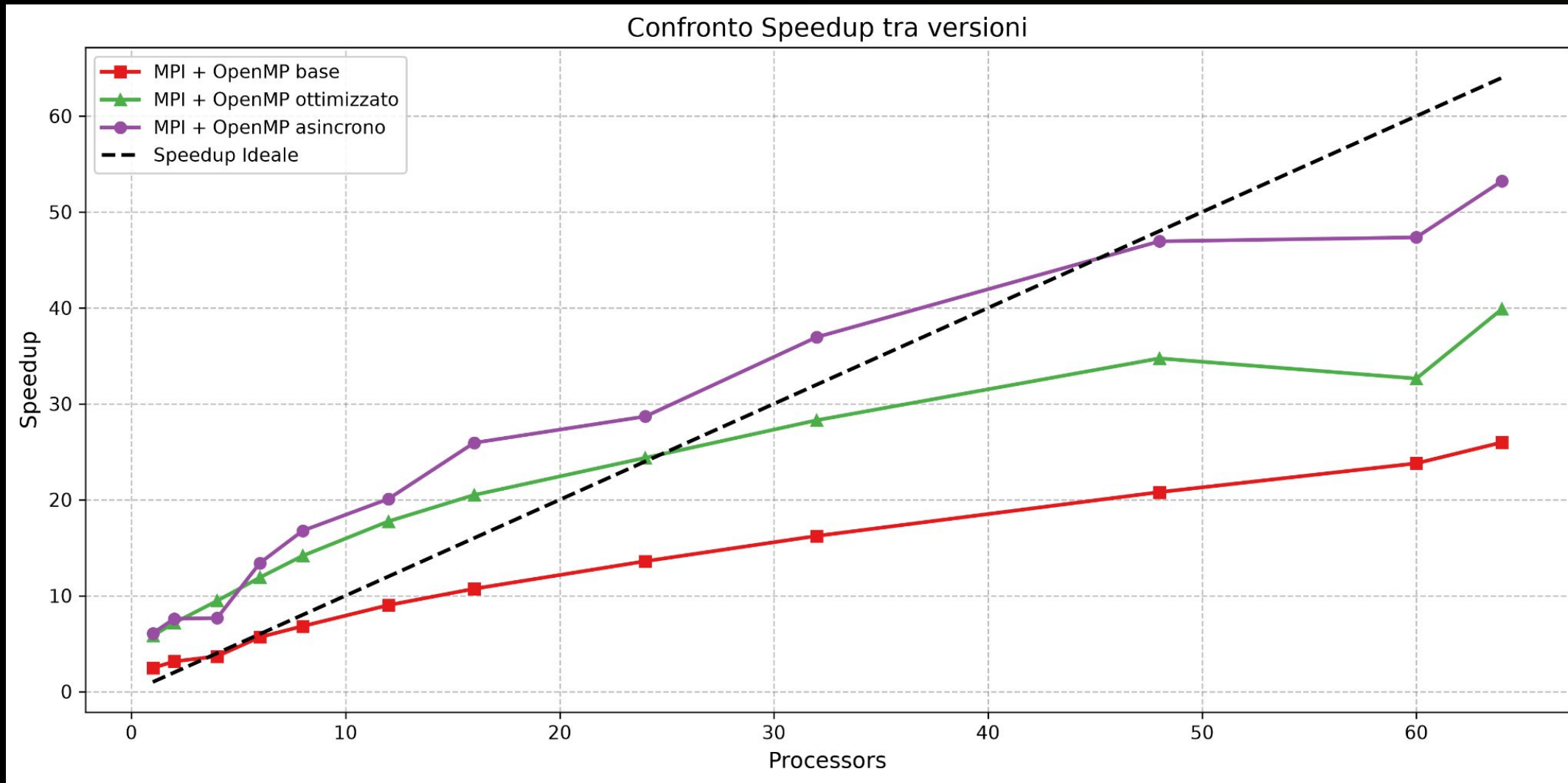
Analisi Performance: Speedup



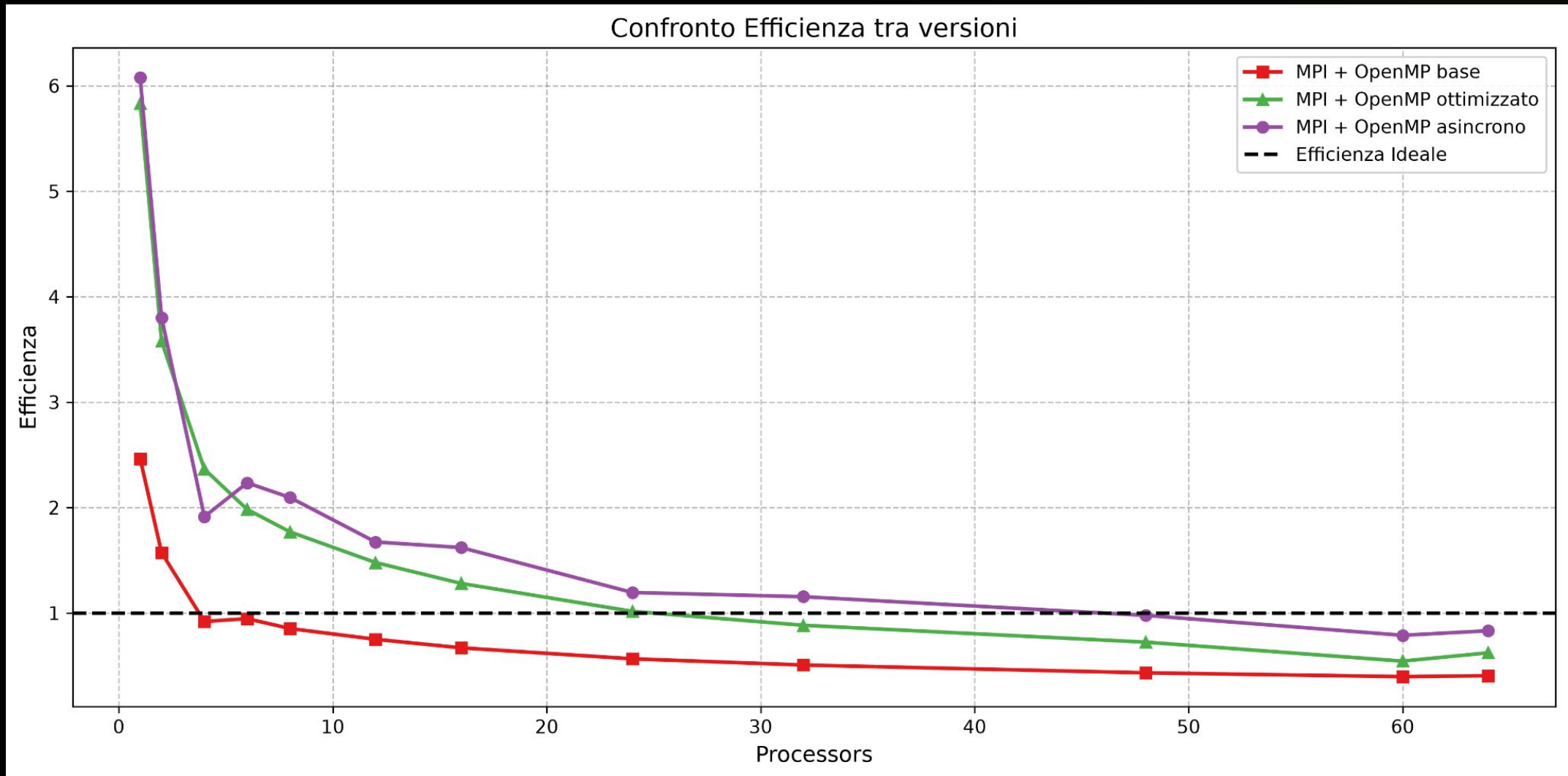
Analisi Performance: Efficienza



Analisi Performance: Speedup



Analisi Performance: Efficienza



Osservazioni Finali

- Reali miglioramenti ottenuti introducendo il multithreading nel contesto MPI → ottimizzazione del bilanciamento MPI/OpenMPI è cruciale.
 - Netto miglioramento dello speedup nella **versione ibrida**, in particolar modo in quella asincrona → mantiene elevati livelli di speedup ed efficienza, prossimi al limite teorico, anche all'aumentare del numero di processori.
 - Andamento superlineare → attribuibile agli effetti della gerarchia di memoria e della cache, favoriti dalla politica di **First Touch** e dalla distinzione tra **computazione locale/remota**.
 - Dimensione del dataset fondamentale → **minore dimensione ⇒ maggiore impatto della comunicazione rispetto al calcolo, con conseguente riduzione della scalabilità.**
-

Domande?

Grazie per l'attenzione.
